DA3408 A3: Spark vs. Ray

The Data Engineering Duel – NYC Taxi Preprocessing

M. Sai Jagadeesh

October 2026

1 Objective and Pipeline

The objective was to implement the same preprocessing pipeline in Apache Spark and Ray Data, use two workers for each framework, and compare execution time, resource usage, and Python UDF overhead on the NYC Yellow Taxi dataset.

The input consists of multiple Yellow Taxi Parquet files and the taxi-zone lookup CSV. Both implementations follow the same logical pipeline:

1. ingest trip files and the zone lookup;
2. normalize timestamps/numeric columns and remove critical nulls;
3. remove duplicate trip records;
4. perform pickup and dropoff location joins;
5. calculate average speed using a custom Python transformation;
6. export the final result to Parquet.

Table 1: Common preprocessing pipeline

Stage	Operation
Ingestion	Read multiple Yellow Taxi Parquet files and taxi-zone CSV.
Cleansing	Normalize timestamps/numeric fields and remove critical nulls.
Duplicates	Remove duplicate trip records using the same definition.
Heavy Join	Join pickup and dropoff location IDs with the zone table.
Feature	Compute average speed from trip distance and duration.
Export	Write the final structured data as Parquet.

2 Cluster and Implementation

Spark: A standalone Spark cluster with one master and two workers was configured manually. The PySpark implementation uses DataFrame operations and a Python UDF for average speed.

Ray: A Ray cluster with one head and two workers was configured manually. The implementation uses `ray.data` for distributed loading, cleaning, joins, Python batch processing, and Parquet output.

The cluster/network setup was performed manually. AI assistance was used for equivalent PySpark/Ray logic and performance reasoning, not for generating benchmark numbers.

For both implementations, the UDF stage was explicitly forced to execute so that its time could be measured separately. The successful Spark run also used `/usr/bin/time -v`; the Ray job was shown as `SUCCEEDED`.

3 Benchmark Results

Table 2: Final benchmark

Framework	Workers	Total (s)	UDF (s)	UDF share
Spark	2	923.69	467.50	50.6%
Ray	2	298.91	182.94	61.2%

The measured end-to-end speedup is

$$\frac{923.69}{298.91} = 3.09\times.$$

Thus Ray completed this workload about 3.09 times faster on the test machine.

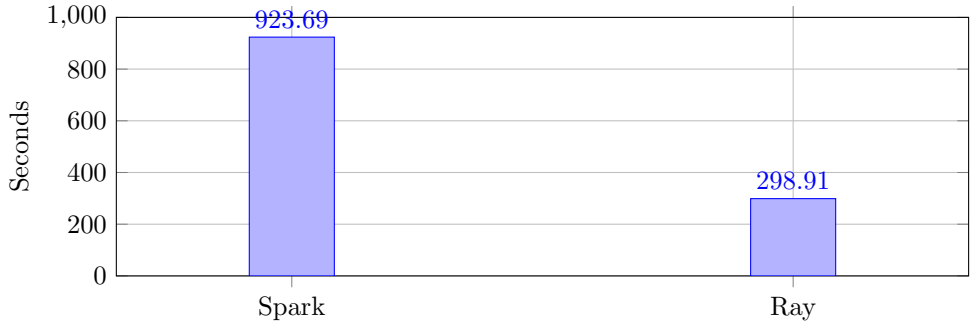


Figure 1: End-to-end preprocessing time.

For the UDF stage,

$$\frac{467.50}{182.94} = 2.56\times.$$

Ray therefore completed the measured UDF stage about 2.56 times faster.



Figure 2: Python UDF execution time.

4 Resource Utilization

The benchmark was run on a single machine with limited RAM and disk space, so the cluster configuration was kept conservative. The retained Spark `/usr/bin/time -v` measurement reported a maximum resident set size of about 568.3 MB for the timed driver process and average CPU usage of 5%. These are driver-process measurements, not cluster-wide peaks.

A cluster-wide CPU/memory time series was not retained for the final Ray run. Therefore no Ray peak value is estimated or fabricated.

Measurement	**Spark**	**Ray**
Timed process max RSS	≈568.3 MB	Not retained
Timed process average CPU	5%	Not retained
Cluster-wide peak CPU/memory	Not retained	Not retained

5 Python UDF Deep-Dive

The derived feature is

$$average_speed_mph = \frac{trip_distance}{duration_hours}.$$

In Spark, the custom Python function executes through the PySpark UDF mechanism, introducing a Python execution boundary within Spark's JVM-based execution model. In Ray, the same calculation is a Python batch transformation in Ray Data, which is closer to its Python-native execution model.

The UDF occupied 50.6% of the measured Spark runtime and 61.2% of the Ray runtime. Although Ray spent a larger fraction of its total time in this stage, its absolute UDF time was much lower (182.94 s versus 467.50 s). This suggests that the Python-native execution model was beneficial for this workload.

6 Performance Tuning Note

During development, the Spark run initially experienced disk and resource pressure. Worker local directories were moved to disk-backed locations and application concurrency was reduced so that the final full-data run could complete successfully.

AI assistance was used to translate equivalent transformations and reason about performance. The cluster/network configuration was performed manually, and the reported benchmark numbers came from the actual completed runs rather than synthetic values.

7 Winner and Framework Choice

Winner for this benchmark: Ray
298.91 s versus 923.69 s, giving a measured 3.09× speedup.

Table 3: Framework choice

Project type	Choice	Reason
AI-first	Ray	Strong result on the Python-heavy workload and natural fit for Python/AI workflows.
BI-first	Spark	Mature choice for structured analytics, DataFrames and SQL-oriented processing.

This does not imply that Ray is always faster. The conclusion is limited to this machine, configuration, dataset, and workload.

8 Evidence and Submission

The final submission should contain the code and actual UI evidence required by the assignment:

- `spark_clean.py` and `ray_clean.py`;
- Spark Master UI showing two active workers;
- Spark application UI on port 4040;
- Ray Dashboard resource view on port 8265;
- Ray Jobs page showing the successful preprocessing job.

The benchmark results reported here are the actual final results:

	Spark	Ray
Total runtime	923.69 s	298.91 s
Python UDF	467.50 s	182.94 s
UDF share	50.6%	61.2%

9 Conclusion

For the completed NYC Taxi experiment, Ray achieved 298.91 s compared with 923.69 s for Spark, an approximately 3.09× speedup. The Python UDF was also faster in Ray (182.94 s versus 467.50 s). Based on this experiment, I would choose Ray for a Python-heavy AI-first workflow and Spark for a BI-first structured analytics workload.